

Project Initialization and Planning Phase

Date	08-07-2026
Team ID	
Project Title	Automated Credit Card Approval Prediction using Machine Learning
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) Report

This proposal describes an **Automated Credit Card Approval Prediction System** that uses Machine Learning to predict whether a credit card application should be **Approved** or **Rejected** based on the applicant's personal and financial information. The system helps banks make faster, more consistent, and data-driven approval decisions while reducing manual effort and processing time.

Project Overview	
Objective	The primary objective of this project is to automate the credit card approval process by analyzing applicant information such as Age, Income, Employment Status, Credit History, Loan Amount, Existing Debt, and other financial details using a trained Machine Learning model. The system aims to improve decision accuracy, reduce manual verification, and provide instant prediction results.
Scope	The project predicts whether a credit card application will be Approved or Rejected using historical applicant data. It provides a simple web-based interface that can be accessed through any web browser without requiring installation. The current scope includes prediction using Machine Learning but does not include real-time bank API integration, document verification, fraud detection, or credit score APIs.

Problem Statement	
Description	Banks receive a large number of credit card applications every day, making manual verification time-consuming and prone to human error. Evaluating multiple financial factors for each applicant can delay approval decisions and lead to inconsistent outcomes. An automated Machine Learning-based prediction system can assist banks by providing faster and more reliable approval recommendations.
Impact	The proposed system helps reduce processing time, improves consistency in decision-making, minimizes manual workload, and enhances customer satisfaction by providing instant credit card approval predictions. It also supports financial institutions in identifying eligible applicants more efficiently.

Proposed Solution	
Approach	The system uses a Random Forest Classifier (or your chosen best-performing model) trained on a historical credit card approval dataset. Before training, missing values are handled, categorical variables are encoded, and numerical features are standardized using StandardScaler . The trained model is integrated with a Flask web application that accepts applicant details, performs preprocessing, and predicts whether the application is approved or rejected
Key Features	❑ Credit Card Approval Prediction Engine: Predicts whether an application is Approved or Rejected using applicant information. ❑ Machine Learning Model: Trained using historical applicant records to improve prediction accuracy. ❑ Interactive Web Interface: Provides Home, About, Credit Card Prediction, and Result pages. ❑ Instant Prediction: Displays approval results immediately after form submission.

Resource Requirements

Resource Type	Description	Specification / Allocation
Hardware		
Computing Resources	Standard development machine — no GPU required, since Logistic Regression and KMeans are lightweight, CPU-based algorithms	Local: Intel Core i3 or above, 4 GB RAM minimum. Hosting: Render free tier (512 MB RAM, shared CPU)
Memory	RAM for model training and Flask application runtime	Training: under 1 GB RAM (small dataset). Production: well within Render's 512 MB free-tier allocation
Storage	Disk space for dataset, trained model, and source code	Dataset (CSV): under 200 KB. Trained model + scaler (.pkl files): a few hundred KB combined. Total repository size: well under 50 MB
Software		
Frameworks	Python web and ML frameworks	Flask — lightweight web framework for the prediction interface. Scikit-learn — Logistic Regression, K-Means, StandardScaler
Libraries	Data science and analysis libraries	pandas, numpy — data preprocessing. matplotlib, seaborn — EDA visualizations. pickle — model persistence
Development Environment	IDE and tools	Jupyter Notebook — model development and EDA. VS Code — Flask application development. Git & GitHub — version control
Data		
Dataset Source	Crop Recommendation Dataset (Kaggle — Smart Agricultural Production Optimizing Engine)	2,200 samples, 22 crop classes (100 each), 7 features (N, P, K, temperature, humidity, pH, rainfall), CSV format, under 200 KB total

Note: this proposal intentionally scopes the project to what has been implemented and tested — a Flask-based web application using classical ML models on a public dataset, deployed on a free-tier host. It does not claim enterprise-scale infrastructure (GPU training, auto-scaling, government dashboards) or unverified impact statistics, since none of that reflects the actual current build.